

Tidal Memory

AI 接入代码手册

正文讲它为什么存在。这个附件只回答三件事：怎么安装、接在哪里、怎样证明它没有乱想起来。

Python ≥ 3.10

SQLite

Vector optional

Apache-2.0

```
# 给维护聊天项目的 coding agent
read("AI_INTEGRATION.md")
inspect(host_application)
integrate(mode="shadow")
prove(recall=True, restraint=True)
```

不要迁移原聊天，不要覆盖旧记忆，不要改变模型人格。Tidal Memory 是派生记忆索引，不是新的聊天数据库。

五分钟启动一个独立记忆库

```
git clone https://github.com/0xblewalker/tidal-memory.git
cd tidal-memory

python -m venv .venv
source .venv/bin/activate
pip install -e .

tidal-memory --db demo.db demo
```

手动写入与验证

```
tidal-memory --db demo.db remember \
  "Rin prefers jasmine tea to coffee." \
  --layer semantic --importance 6

tidal-memory --db demo.db impression chat-001 \
  "They talked about rainy walks; the mood was easy."

tidal-memory --db demo.db context --conversation-id chat-002
tidal-memory --db demo.db recall --force \
  "Do you remember what Rin likes to drink?"
```

生产环境：数据库放在仓库外的受保护路径；不要提交 *.db、环境变量、聊天原文或真实记忆导出。

接入聊天链路 只需要三个位置

```
from tidal_memory import RecallPolicy, TidalMemory

memory = TidalMemory(
    "/protected/path/memory.db",
    policy=RecallPolicy(
        trigger="balanced",
        association="direct_plus_one_hop",
        max_items=2,
        max_chars=900,
        repeat_cooldown_turns=3,
    ),
)
```

-
- 01 新窗口首轮：memory.opening_context(conversation_id)
注入稳定事实与低分辨率印象，仅一次。
-
- 02 每次用户发言：memory.recall(user_text)
只有策略允许时才返回旧细节。
-
- 03 窗口真正结束：memory.close_window(id, active_messages)
生成一条印象，并可抽取长期事实。

两个注入块都应是临时上下文，不写回可见聊天记录。当前用户消息永远比旧记忆更可信。

完整的请求组装 应该长这样

```
def build_model_messages(conversation_id, history, user_text):
    messages = list(history)

    if not history:
        opening = memory.opening_context(conversation_id)
        if opening:
            messages.append({
                "role": "system",
                "content": (
                    '<memory source="opening" resolution="low">\n'
                    + opening + "\n</memory>"
                ),
            })

    detail = memory.recall(user_text)
    if detail:
        messages.append({
            "role": "system",
            "content": (
                '<memory source="recall" resolution="exact">\n'
                + detail + "\n</memory>"
            ),
        })

    messages.append({"role": "user", "content": user_text})
    return messages
```

缓存前缀要稳定：不要把时间戳、调试计数、随机 ID 放进 opening block。每轮变化的 recalled detail 应位于稳定前缀之后。

让任何模型负责 写印象与提取事实

模糊印象

保留话题、关系氛围和至多一个未解决方向。删除精确数字、逐字引用、工具细节与私密细节。

长期事实

只保存跨窗口仍可能有用的偏好、承诺、关系变化与项目状态。笑话和临时情绪不晋升。

```
def vague_writer(messages):  
    # 可调用 Claude、GPT、Gemini 或本地模型  
    return "他们晚上一起修了些东西；整体轻松亲密。"  
  
def extract_facts(messages):  
    return [{  
        "summary": "Rin prefers jasmine tea.",  
        "layer": "semantic",  
        "importance": 6,  
        "tags": "preference,drink",  
    }]  
  
memory = TidalMemory(  
    "memory.db",  
    impression_writer=vague_writer,  
    fact_extractor=extract_facts,  
)
```

也可以传入 `relevance_verifier(query, candidates)`，让廉价模型在最终注入前剔除词面相似、实际无关或已经过时的候选。

最容易出错的 不是召回，是重写

场景	必须发生	禁止发生
编辑旧消息	重新从当前有效分支组装请求	把被替换内容继续留给模型
重新生成	排除被放弃的 assistant 回复	为废弃分支生成窗口印象
彻底重写	取消旧分支待执行的提取任务	重复写入同一长期事实
页面断开	延迟并去重窗口收束任务	把一次断线当作永久关窗

先旁路运行

```
TIDAL_MEMORY_INJECT=0
```

- 正常写入一份全新的记忆数据库。
- 执行召回判断，但不把结果注入模型。
- 日志只记录 ID、分数、来源、字符数与抑制原因，不记录私密正文。
- 观察误命中、漏命中和重复命中，再决定是否开启注入。

复制这一段， 让 AI 给自己安装

阅读仓库中的 AI_INTEGRATION.md，检查我现有聊天项目的消息存储、请求组装、流式结束、新窗口、编辑、重新生成和定时任务。先以旁路模式接入 Tidal Memory，不要删除、覆盖或迁移已有聊天与记忆。修改前备份；完成库内测试和真实 API 的冷启动、明确召回、编辑/重生成测试后，向我报告钩子位置、误命中样本、成本、回滚方法，再由我决定是否开启正式注入。

开启前最低验收

- ✓ 寒暄不翻旧账
- ✓ 明确追问能命中
- ✓ 注入不超过预算
- ✓ 同一记忆有冷却
- ✓ 新窗口只带模糊印象

- ✓ 重写不泄露旧分支
- ✓ 一窗最多一条印象
- ✓ 周/月合并保持有界
- ✓ 记忆故障不阻断聊天
- ✓ 原数据保持不变

github.com/0xblewalker/tidal-memory

完整接口、测试、Retriever 适配方式与隐私边界均在仓库中。这个 PDF 是接入速查表，不替代 AI_INTEGRATION.md。